HU-01 / PRÁCTICA DE LABORATORIO

Microservicio CRUD con MongoDB

Calidad de código y pruebas automatizadas

Implementación RESTful de incidencias de sensores

REPORTE TÉCNICO

ADAN GIOVANNI RAMIREZ ERIVES

Asignatura: Desarrollo Web Integral
Unidad II · Modalidad individual
Fecha del reporte: 13 de noviembre de 2026

12 pruebas aprobadas · 94.87 % de líneas

Quality Gate de SonarQube: APROBADO

Repositorio público: github.com/JustPolar1/telemetria
Rama de entrega: feature/incidents-mongodb-hu01

1. Microservicio y persistencia NoSQL

Se implementó **Incident Service**, un microservicio REST independiente para administrar incidencias de sensores. Utiliza Java 21, Spring Boot 4.1.1 y Spring Data MongoDB. Los servicios previos de parcelas y telemetría conservan su persistencia PostgreSQL; el cumplimiento de HU-01 corresponde al nuevo módulo.

Arquitectura y responsabilidad

Swagger UI / cliente HTTP → IncidentController → IncidentService → IncidentRepository → MongoDB. El controlador valida entradas; el servicio aplica la lógica de negocio; el repositorio persiste documentos en la colección **incidents**. Las excepciones se traducen mediante IncidentExceptionHandler.

Campo	Regla o responsabilidad
id / folio	Identificador MongoDB y folio UUID generado, con índice único.
sensorId	Obligatorio; hasta 100 caracteres alfanuméricos, guiones o guiones bajos.
titulo / descripcion	Obligatorios; máximo 150 y 2000 caracteres. Se recortan los extremos.
estado	ABIERTA, EN_REVISION o RESUELTA; obligatorio.
fechaCreacion	Fecha UTC generada por el servicio; se conserva al actualizar.

Contrato REST

Base: **/api/v1/incidents**. El listado acepta page ≥ 0 y size entre 1 y 100.

Método	Ruta relativa	Resultado esperado
POST	/	201 Created + Location
GET	/	200 + página de incidencias
GET	/{id}	200 o 404
PUT	/{id}	200 o 404
DELETE	/{id}	204 o 404

sensorId es una referencia del dominio. El CRUD no requiere consultar a Telemetry Service para comprobar la existencia del sensor.

2. Excepciones, pruebas y cobertura

Situación	Tratamiento y HTTP
Recurso inexistente	IncidentNotFoundException → 404.
Folio duplicado	DuplicateIncidentException → 409; índice único en MongoDB.
Entrada inválida	Validación de campos, JSON, enum y paginación → 400.
Fallo de persistencia	DataAccessException → 503 con mensaje genérico.

El manejador global usa **@RestControllerAdvice** y devuelve JSON con la clave **error** y un mensaje descriptivo. El folio se genera en el servidor: repetir sensor o título está permitido; el conflicto 409 protege la unicidad del folio.

Pruebas ejecutadas

Suite	Casos	Evidencia funcional
IncidentServiceTests	7	JUnit y Mockito: altas, consultas, listado, actualización, eliminación y errores.
IncidentControllerTests	4	MockMvc: contrato HTTP, JSON, validaciones y errores de negocio/persistencia.
IncidentMongoIT	1	Spring Boot + MongoDB 8.0 real en Testcontainers: CRUD, índice único y contrato OpenAPI.

Resultado: **12 pruebas, 0 fallos, 0 errores y 0 omitidas**. La integración comprueba la persistencia de las modificaciones y que consultar un recurso eliminado devuelva 404. Los contenedores temporales se retiran al finalizar.

JaCoCo

El reporte registra **37 de 39 líneas cubiertas (94.87 %)**. Se configuraron reglas de al menos 80 % para líneas y ramas. No se detectaron ramas medibles (0 de 0): se reportan como **n/a**, no como 100 %. El 98 % visible en la columna principal del HTML corresponde a instrucciones, no a líneas.

El registro completo está en evidence/verify.txt. El HTML portable y XML de cobertura se conservaron en evidence/jacoco/. La ejecución terminó con BUILD SUCCESS y aprobó las reglas de cobertura.

3. SonarQube y entrega reproducible

Se ejecutó SonarQube Community Build 26.11.0.134175 sobre Java principal, pruebas y pom.xml, con el perfil activo Sonar way comprehensive. Se importaron el XML de JaCoCo y los reportes de pruebas unitarias y de integración.

Condición global de HU-01	Umbral	Resultado
Cobertura	≥ 80 %	94.9 %
Bugs	0	0
Vulnerabilidades	0	0
Code Smells	0	0

Quality Gate: HU-01 Incidencias, estado OK / Passed. Las condiciones globales de la tabla se evaluaron y aprobaron. La configuración también conserva las condiciones estándar del servidor para código nuevo.

Se registraron 0 Security Hotspots y 0.0 % de duplicación. Los resultados corresponden al alcance del analizador y perfil utilizados; no constituyen un análisis de dependencias. Los tokens temporales de análisis se revocaron al finalizar.

Ejecución desde la raíz del repositorio

```
docker compose -f incident-service/compose.yaml up --build -d
mvn -f incident-service/pom.xml clean verify
```

Swagger: http://localhost:8083/swagger-ui/index.html
MongoDB de desarrollo: localhost:27018, base incidents. Los puertos se publican en loopback para el laboratorio.

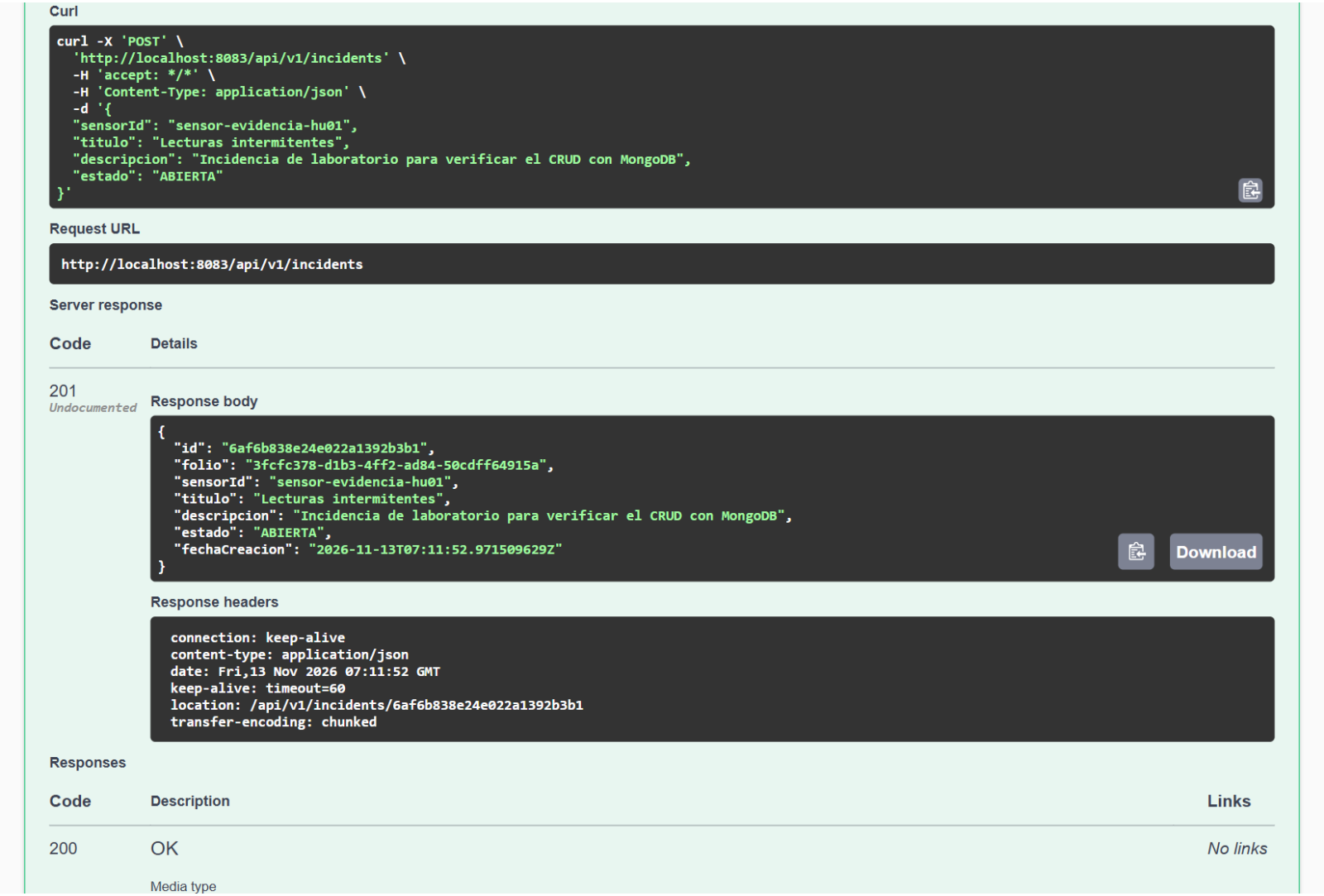
Con SonarQube iniciado y un SONAR_TOKEN válido configurado externamente:
`mvn -f incident-service/pom.xml sonar:sonar "-Dsonar.qualitygate.wait=true"`
SONAR_HOST_URL debe apuntar al servidor; en esta práctica se usó http://localhost:9000.

Código y README: abrir módulo en la rama de entrega. Los cambios se registraron en commits de implementación, pruebas, despliegue y documentación, en una rama feature creada desde Develop.

Referencias: Spring Boot / MongoDB · springdoc / Swagger UI · JaCoCo check · SonarQube.

4. Crear incidencia - POST 201

Solicitud enviada desde Swagger UI. El servidor devuelve ID, folio y fecha de creación; la incidencia se guarda en MongoDB.



Curl

```
curl -X 'POST' \
  'http://localhost:8083/api/v1/incidents' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "sensorId": "sensor-evidencia-hu01",
  "titulo": "Lecturas intermitentes",
  "descripcion": "Incidencia de laboratorio para verificar el CRUD con MongoDB",
  "estado": "ABIERTA"
}'
```

Request URL

```
http://localhost:8083/api/v1/incidents
```

Server response

Code	Details
201 *Undocumented*	Response body

```
{
  "id": "6af6b838e24e022a1392b3b1",
  "folio": "3fcfc378-d1b3-4ff2-ad84-50cdff64915a",
  "sensorId": "sensor-evidencia-hu01",
  "titulo": "Lecturas intermitentes",
  "descripcion": "Incidencia de laboratorio para verificar el CRUD con MongoDB",
  "estado": "ABIERTA",
  "fechaCreacion": "2026-11-13T07:11:52.971509629Z"
}
```

Download

Response headers

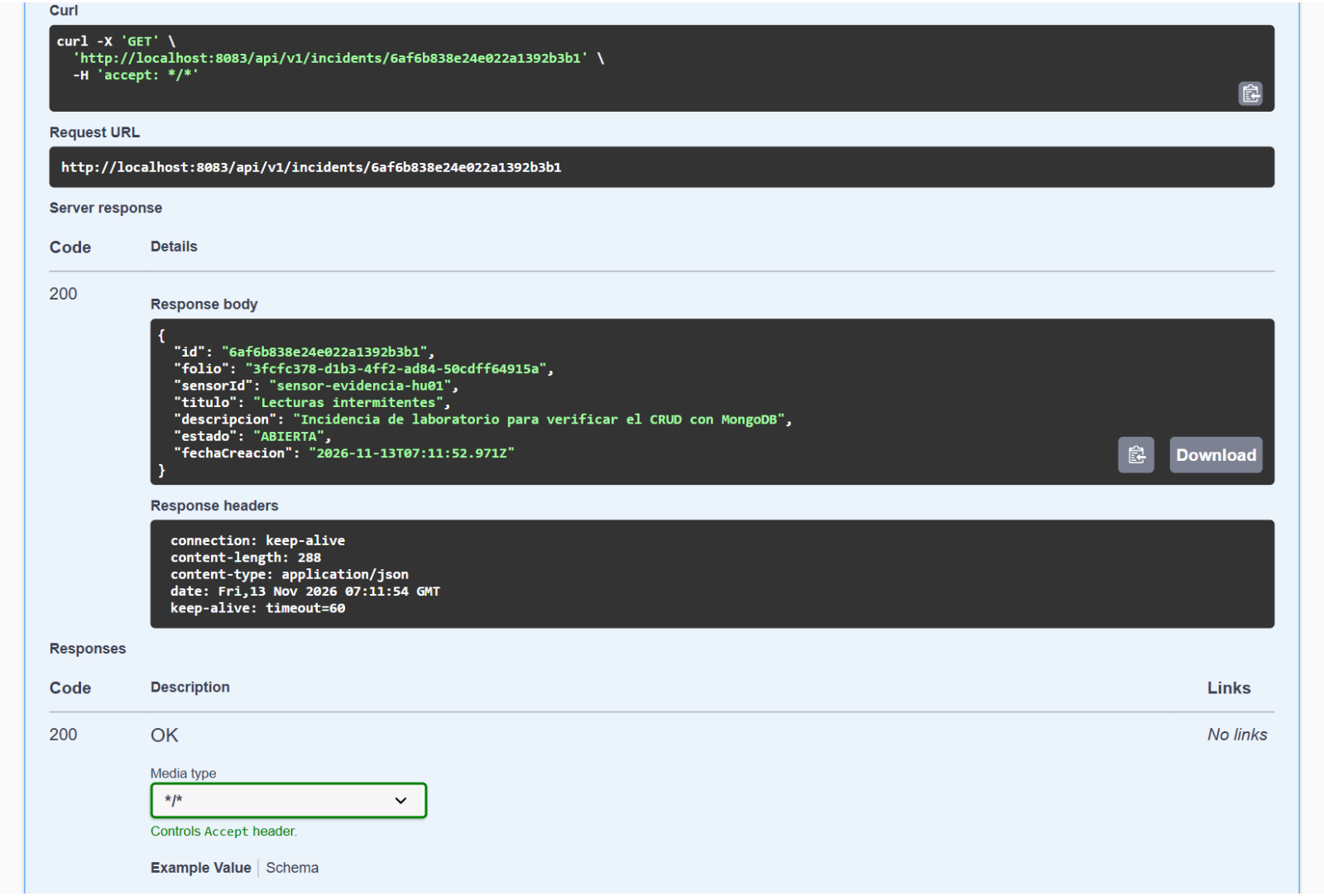
```
connection: keep-alive
content-type: application/json
date: Fri,13 Nov 2026 07:11:52 GMT
keep-alive: timeout=60
location: /api/v1/incidents/6af6b838e24e022a1392b3b1
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK Media type	*No links*

5. Consultar incidencia - GET 200

Consulta del mismo ID creado en la página anterior. La respuesta recupera el documento persistido.



6. Listar incidencias - GET 200

Listado paginado que contiene el documento creado; se muestran content y los metadatos de paginación.

Curl

```
curl -X 'GET' \
  'http://localhost:8083/api/v1/incidents?page=0&size=20' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8083/api/v1/incidents?page=0&size=20
```

Server response

Code	Details
200	Response body


```
{
  "content": [
    {
      "id": "6af6b838e24e022a1392b3b1",
      "folio": "3fcfc378-d1b3-4ff2-ad84-50cdff64915a",
      "sensorId": "sensor-evidencia-hu01",
      "titulo": "Lecturas intermitentes",
      "descripcion": "Incidencia de laboratorio para verificar el CRUD con MongoDB",
      "estado": "ABIERTA",
      "fechaCreacion": "2026-11-13T07:11:52.971Z"
    }
  ],
  "totalElements": 1,
  "totalPages": 1,
  "number": 0,
  "size": 20
}
```


Download

Response headers

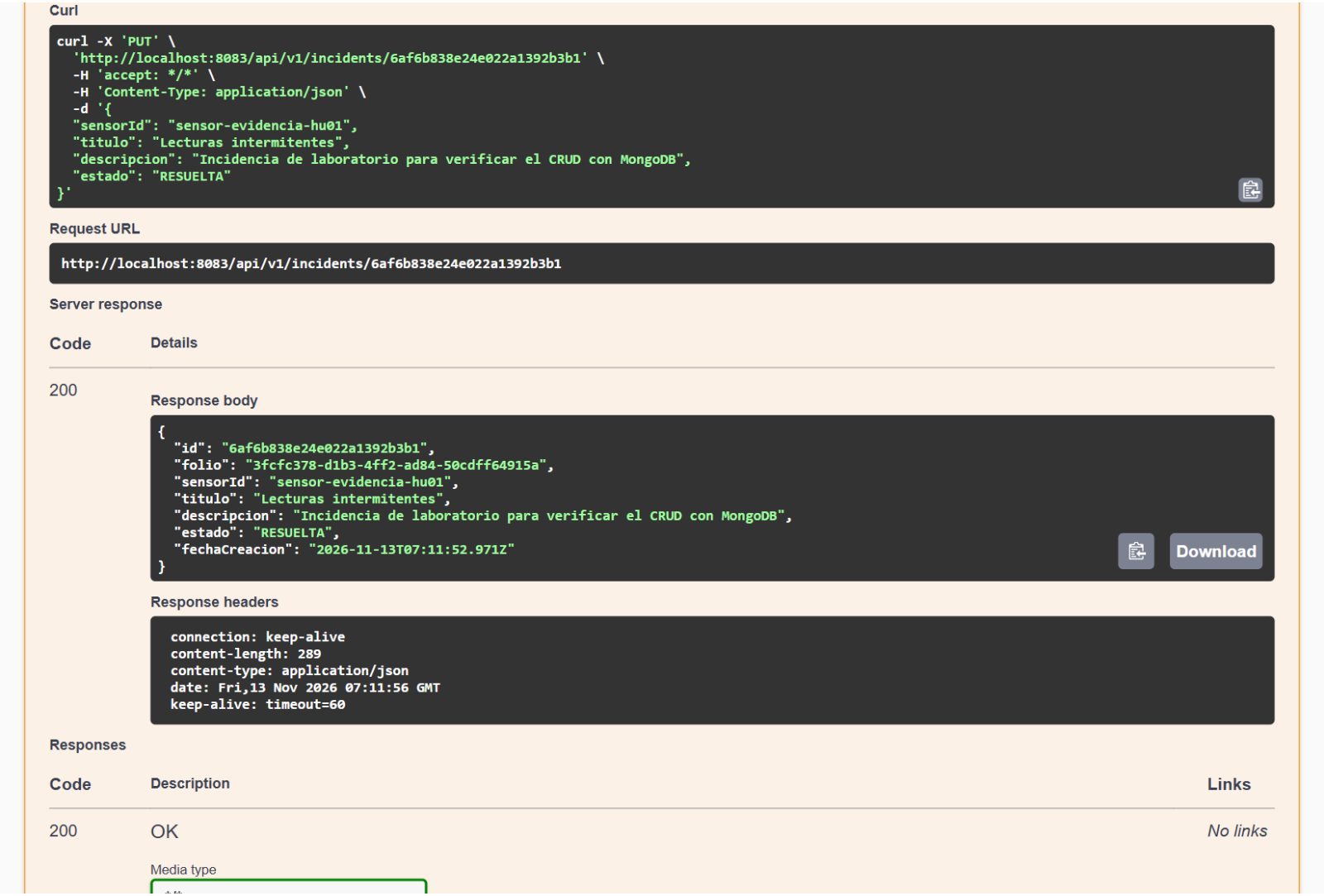
```
connection: keep-alive
content-length: 356
content-type: application/json
date: Fri,13 Nov 2026 07:11:55 GMT
keep-alive: timeout=60
```

Responses

Code	Description	Links
200	OK	*No links*

7. Actualizar incidencia - PUT 200

Se reemplazan los campos editables y se cambia el estado a RESUELTA, conservando ID, folio y fecha de creación.



Curl

```
curl -X 'PUT' \
  'http://localhost:8083/api/v1/incidents/6af6b838e24e022a1392b3b1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "sensorId": "sensor-evidencia-hu01",
  "titulo": "Lecturas intermitentes",
  "descripcion": "Incidencia de laboratorio para verificar el CRUD con MongoDB",
  "estado": "RESUELTA"
}'
```

Request URL

```
http://localhost:8083/api/v1/incidents/6af6b838e24e022a1392b3b1
```

Server response

Code	Details
200	Response body

```
{
  "id": "6af6b838e24e022a1392b3b1",
  "folio": "3fcfc378-d1b3-4ff2-ad84-50cdff64915a",
  "sensorId": "sensor-evidencia-hu01",
  "titulo": "Lecturas intermitentes",
  "descripcion": "Incidencia de laboratorio para verificar el CRUD con MongoDB",
  "estado": "RESUELTA",
  "fechaCreacion": "2026-11-13T07:11:52.971Z"
}
```

Download

Response headers

```
connection: keep-alive
content-length: 289
content-type: application/json
date: Fri,13 Nov 2026 07:11:56 GMT
keep-alive: timeout=60
```

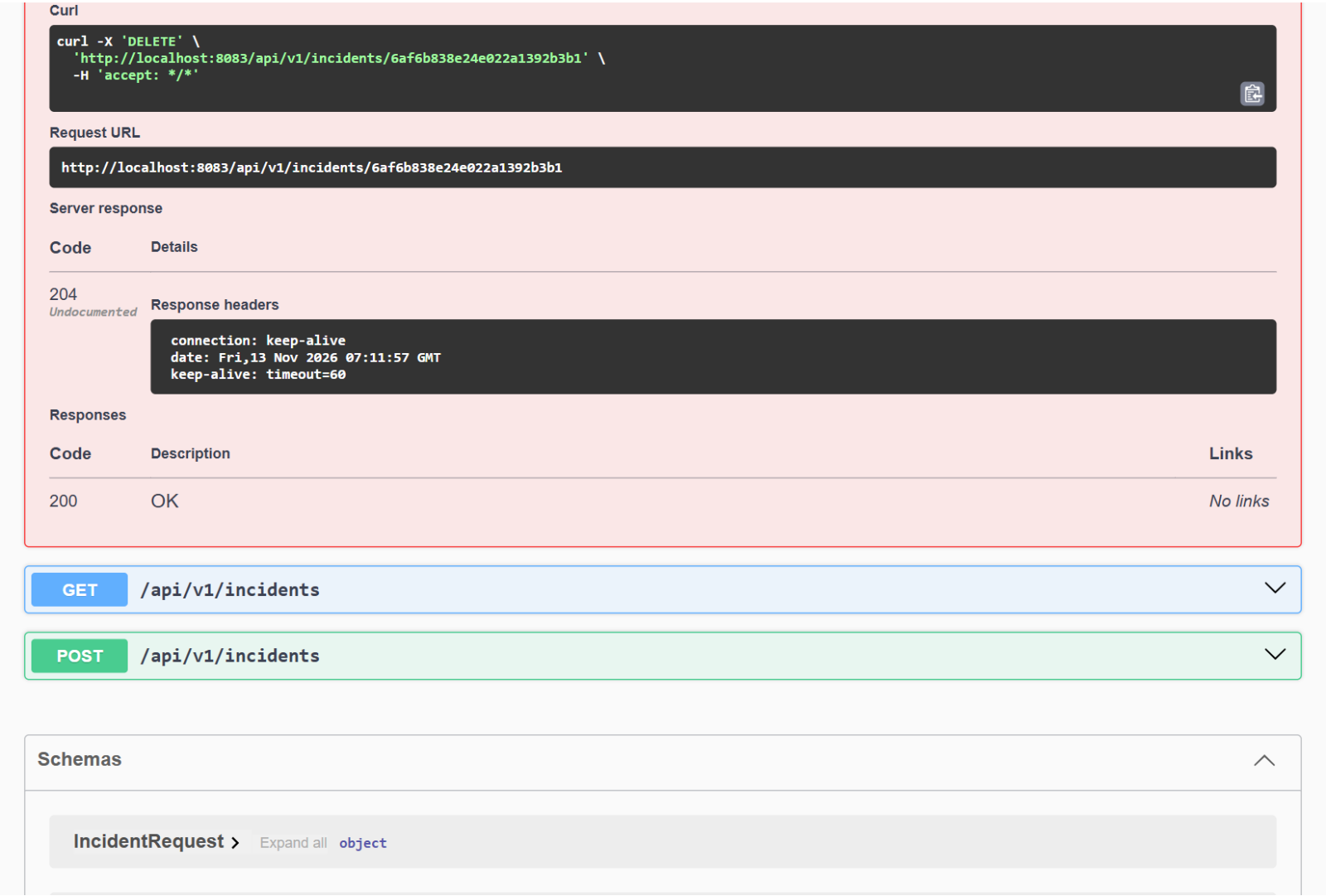
Responses

Code	Description	Links
200	OK	*No links*

Media type

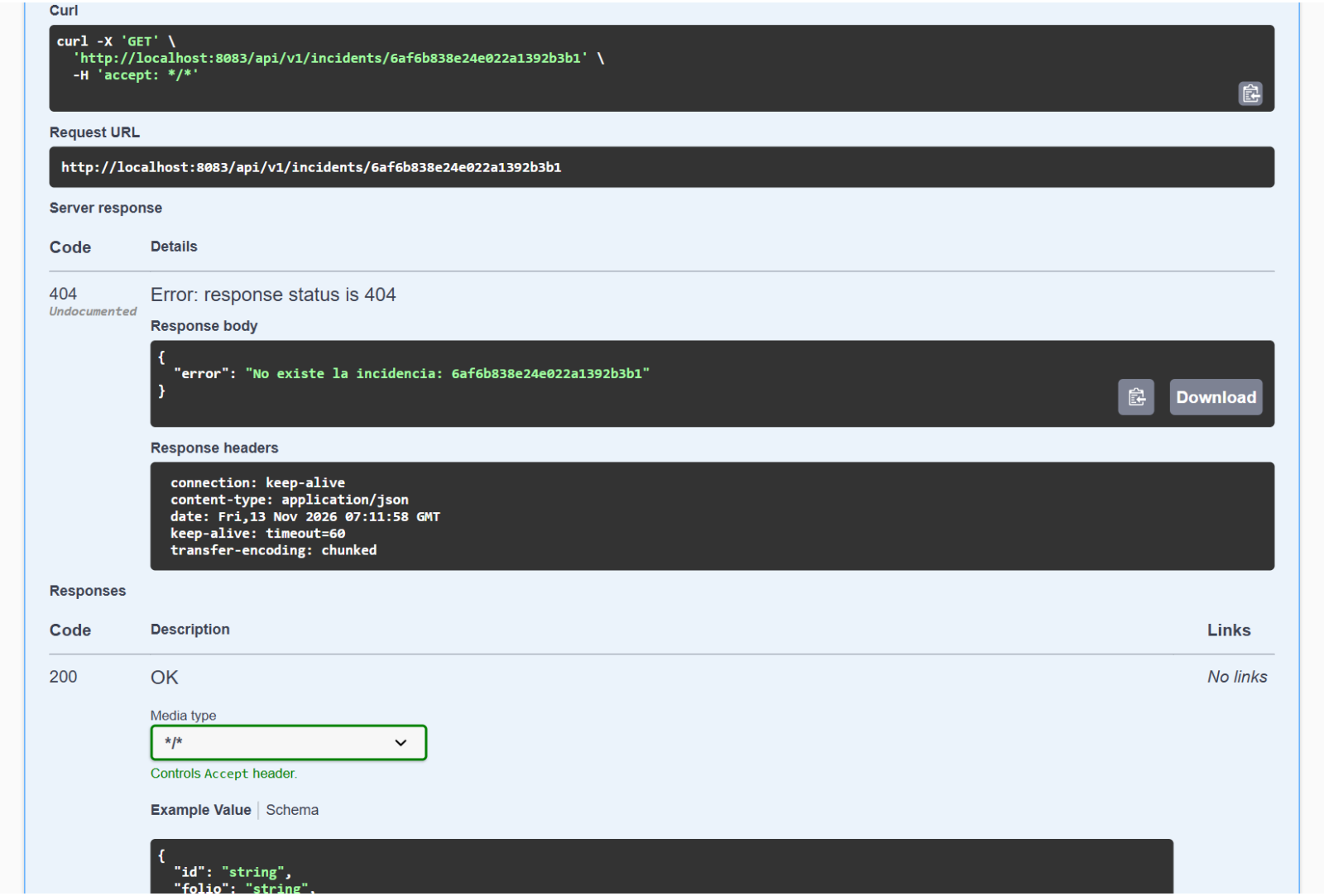
8. Eliminar incidencia - DELETE 204

El servidor confirma la eliminación del documento sin cuerpo de respuesta.



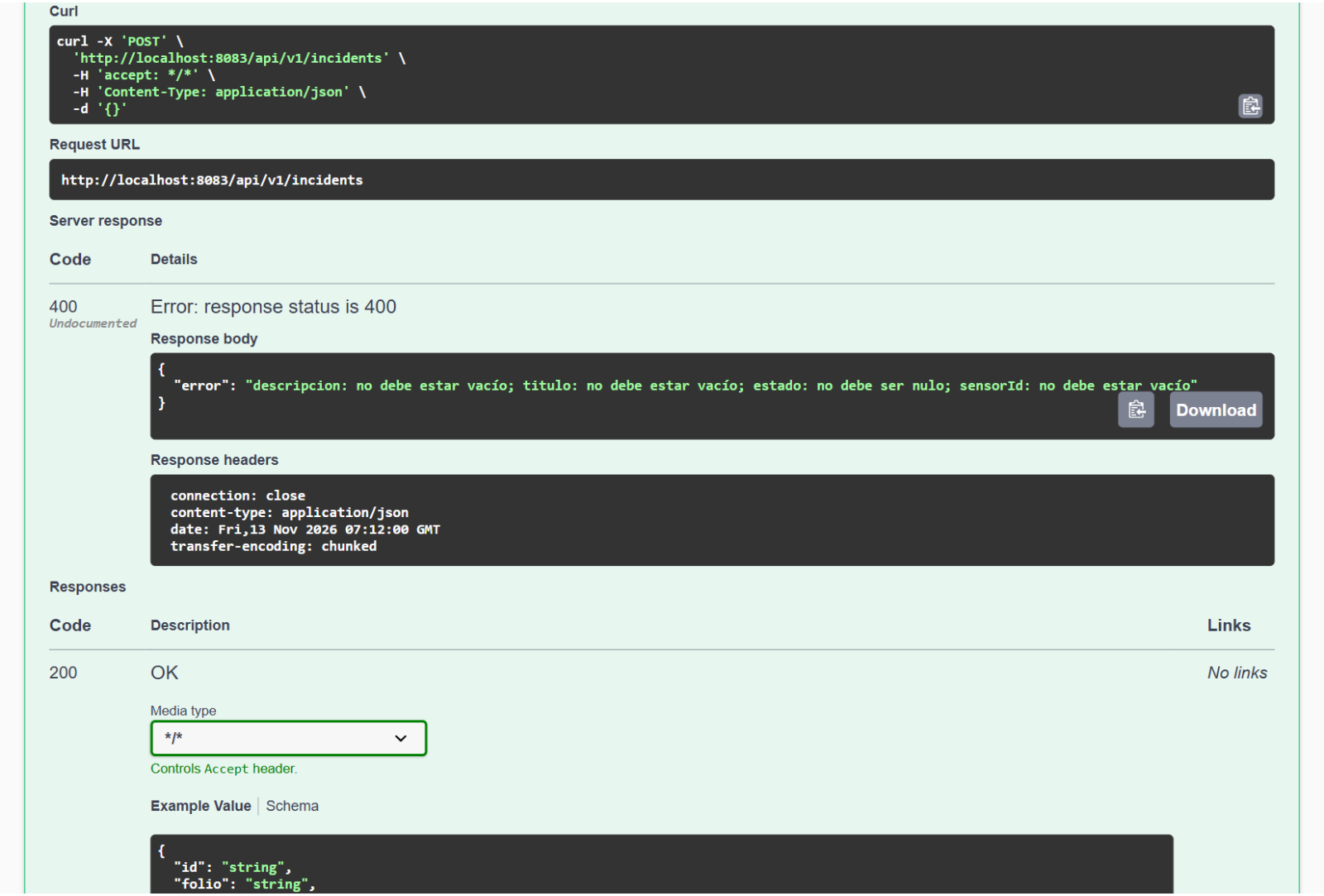
9. Recurso inexistente - GET 404

Después de eliminar el documento, consultar el mismo ID devuelve un error JSON descriptivo con estado 404.



10. Validar entrada - POST 400

Un cuerpo vacío es rechazado por las validaciones del modelo y el manejador global devuelve HTTP 400.



11. Ejecución de pruebas

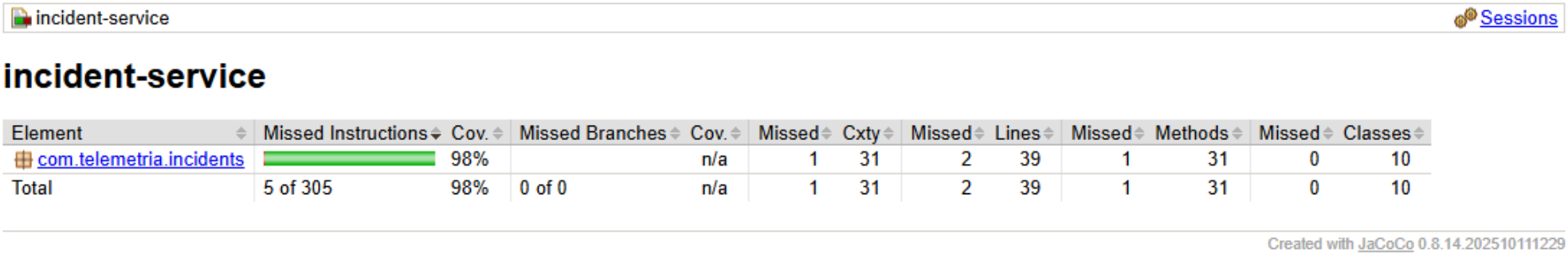
Captura del registro real de Maven al finalizar la integración y la comprobación de cobertura. El registro íntegro está conservado en evidence/verify.txt.

```
5.8.1.jar:na]
        at com.mongodb.internal.connection.DefaultServerMonitor$ServerMonitor.run(DefaultServerMonitor.java:203) ~[mongodb-driver-core-5.8.1.jar:na]

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 39.60 s -- in com.telemetria.incidents.IncidentMongoIT
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- failsafe:3.5.6:verify (default) @ incident-service ---
[INFO]
[INFO] --- jacoco:0.8.14:report (report-and-check) @ incident-service ---
[INFO] Loading execution data file C:\Users\Lenovo\Documents\utch\9o cuatrimestre\Desarrollo Web Integral\Unidad II\Telemetr◆a_b◆sica\incident-
service\target\jacoco.exec
[INFO] Analyzed bundle 'incident-service' with 10 classes
[INFO]
[INFO] --- jacoco:0.8.14:check (report-and-check) @ incident-service ---
[INFO] Loading execution data file C:\Users\Lenovo\Documents\utch\9o cuatrimestre\Desarrollo Web Integral\Unidad II\Telemetr◆a_b◆sica\incident-
service\target\jacoco.exec
[INFO] Analyzed bundle 'incident-service' with 10 classes
[INFO] All coverage checks have been met.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:56 min
[INFO] Finished at: 2026-11-13T00:02:24-06:00
[INFO] ------------------------------------------------------------------------
```

12. Reporte HTML de JaCoCo

Reporte generado por clean verify: 2 líneas sin cubrir de 39 (37 cubiertas, 94.87 %). Las ramas aparecen como n/a porque no hay ramas medibles.



incident-service

Sessions

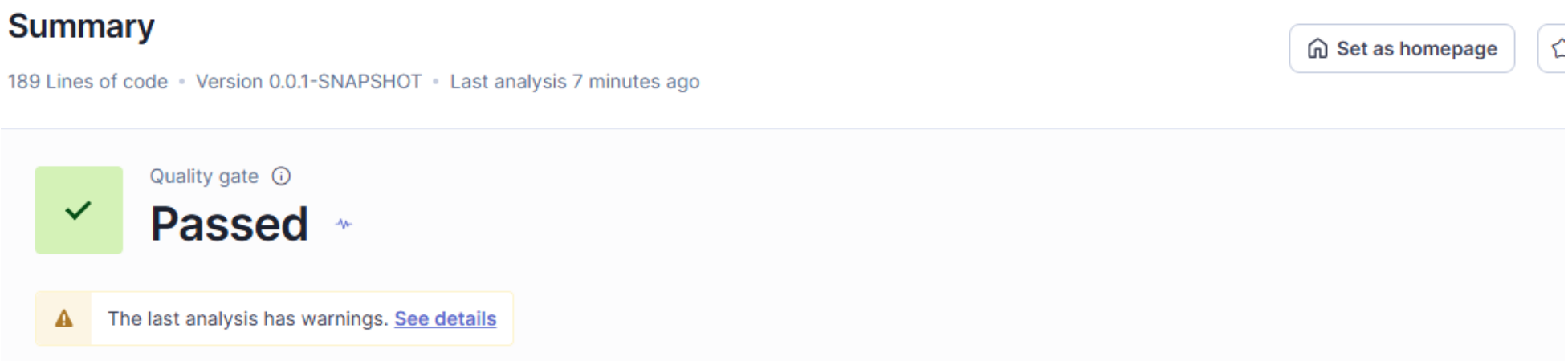
incident-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.telemetria.incidents		98%		n/a	1	31	2	39	1	31	0	10
Total	5 of 305	98%	0 of 0	n/a	1	31	2	39	1	31	0	10

Created with JaCoCo 0.8.14.202510111229

13. Panel de SonarQube - Quality Gate aprobado

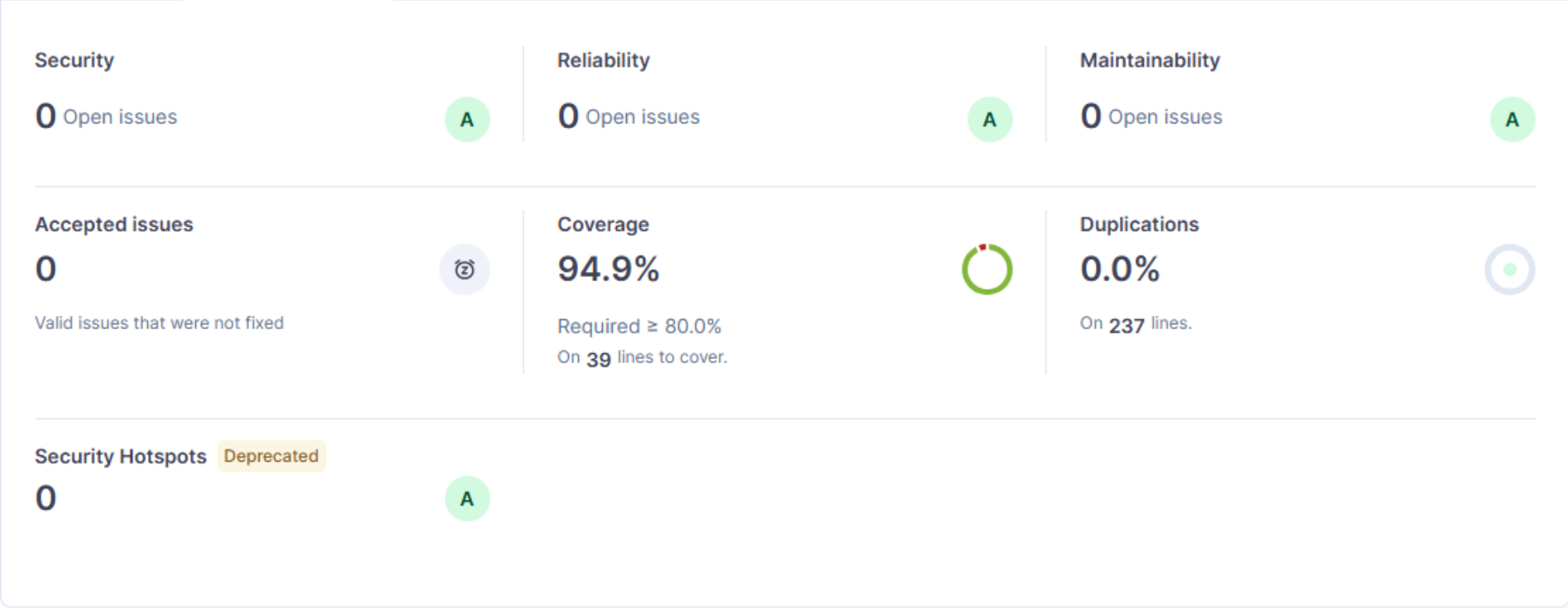
Fragmentos de la misma captura del panel global: Quality Gate Passed, cobertura 94.9 % y cero problemas abiertos de seguridad, fiabilidad y mantenibilidad. La captura íntegra se conserva en evidence/sonar-dashboard.png.



Condición global	Umbral exigido	Valor evaluado
Cobertura	Al menos 80 %	94.9 %
Bugs	0	0
Vulnerabilidades	0	0
Code Smells	0	0

14. SonarQube - métricas globales

Detalle de Overall Code en la misma captura: cero problemas abiertos, cobertura de 94.9 % y duplicación de 0.0 %. El JSON original de métricas se conserva en evidence/sonar-results.json.



Repositorio público

https://github.com/JustPolar1/telemetria